

Protein Circuit Tracing via Cross-layer Transcoders

Darin Tsui¹ Kunal Talreja¹ Daniel Saeedi¹ Amirali Aghazadeh¹

Abstract

Protein language models (pLMs) have emerged as powerful predictors of protein structure and function. However, the computational circuits underlying their predictions remain poorly understood. Recent mechanistic interpretability methods decompose pLM representations into interpretable features, but they treat each layer independently and thus fail to capture cross-layer computation, limiting their ability to approximate the full model. We introduce ProtoMech, a framework for discovering computational circuits in pLMs using *cross-layer transcoders* that learn sparse latent representations jointly across layers to capture the model’s full computational circuitry. Applied to the pLM ESM2, ProtoMech recovers 82–89% of the original performance on protein family classification and function prediction tasks. ProtoMech then identifies compressed circuits that use <1% of the latent space while retaining up to 79% of model accuracy, revealing correspondence with structural and functional motifs, including binding, signaling, and stability. Steering along these circuits enables high-fitness protein design, surpassing baseline methods in more than 70% of cases. These results establish ProtoMech as a principled framework for protein circuit tracing.

1. Introduction

Protein language models (pLMs) have driven rapid progress in the biosciences by learning rich statistical representations from large protein sequence databases. They now achieve strong performance across a broad range of downstream tasks, including 3D structure and function prediction (Lin et al., 2023; Hayes et al., 2025; Bhatnagar et al., 2025). These results suggest that pLMs may capture latent structural and functional motifs governing protein se-

quences (Rives et al., 2021; Tsui et al., 2025a; Tsui & Aghazadeh, 2024). However, the internal computational pathways, or *circuits*, responsible for these predictions remain opaque, poorly understood, and difficult to extract.

Recent progress in mechanistic interpretability, particularly through sparse autoencoders (SAEs) (Templeton et al., 2024; Gao et al., 2025), has enabled the decomposition of pLM hidden states into interpretable features (Adams et al., 2025; Simon & Zou, 2025; Walton et al., 2025a; Gujral et al., 2025; Nainani et al., 2025; Parsan et al., 2025). Steering these features has been shown to generate protein sequences with specific functional attributes (Tsui et al., 2025b; Corominas et al., 2025; Garcia & Ansuini, 2025). However, SAEs provide only a representational factorization and do not capture the layer-to-layer transformations that capture the model’s computation. Consequently, they cannot recover the full circuitry responsible for pLM predictions. Recovering such circuits requires a *replacement model* that faithfully emulates the internal computation of the original network.

In the natural language processing literature, recent efforts have aimed to construct replacement models using *transcoders* (Dunefsky et al., 2024). In contrast to SAEs, which factorize representations, transcoders approximate the functional mapping of individual transformer MLP layers by passing activations through a sparse latent bottleneck. Composing these approximations across layers yields per-layer transcoders (PLTs), which seek to reconstruct model computation from locally sparse surrogates. Yet this construction remains inherently local: approximating each layer in isolation neglects the accumulation of context and computation across depth, leading to degraded representations and unreliable circuit recovery (Ameisen et al., 2025).

In this work, we bridge the gap between interpretable feature recovery and mechanistic circuit discovery in pLMs. We introduce ProtoMech, a framework for uncovering computational circuits in pLMs using *cross-layer transcoders* (CLTs) (Ameisen et al., 2025). Similar to standard transcoders, CLTs learn an input–output mapping for each transformer MLP layer. However, rather than relying on isolated layerwise approximations, CLTs compute each layer’s output as a function of the sparse latent variables from all preceding layers. By explicitly modeling these cross-layer dependencies, ProtoMech constructs a replacement model

¹School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA. Correspondence to: Amirali Aghazadeh <amiralia@gatech.edu>.

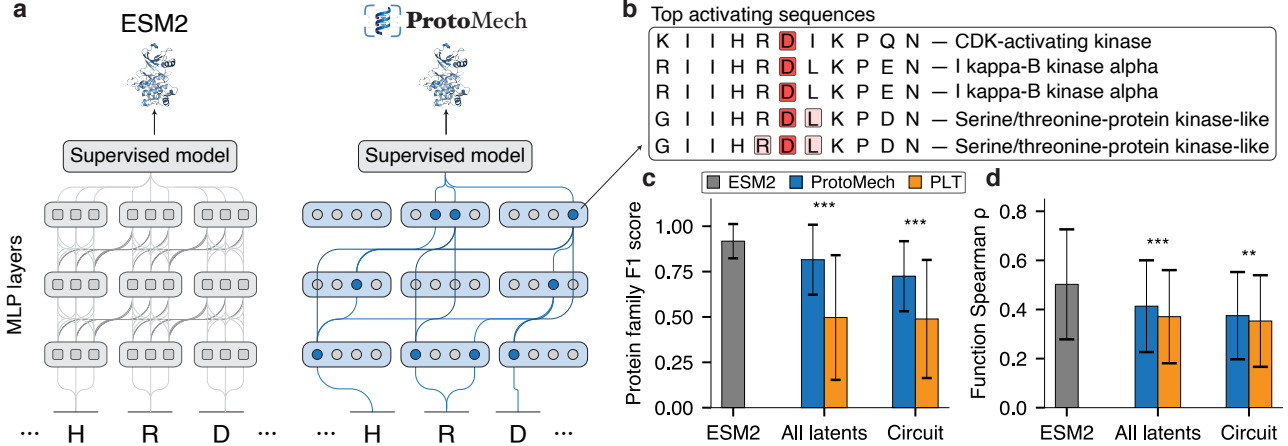


Figure 1. ProtoMech serves as a replacement model for ESM2. **a**, Schematic of the circuit discovery process. ProtoMech identifies a circuit of interpretable latents (blue) that traces and approximates the behavior of ESM2 on downstream tasks. **b**, Example of top activating sequences in Swiss-Prot for a specific latent (L3/1918), which detects the conserved HRD catalytic motif found in protein kinases. On **c**, protein family classification and **d**, function prediction downstream tasks, ProtoMech outperforms PLT baselines.

that more faithfully reproduces the internal computation of the pLM (Fig. 1), enabling direct identification of the circuits that govern its predictions.

Our contributions are as follows:

- We develop ProtoMech, a framework for discovering and analyzing computational circuits within and across transformer layers in pLMs. Applied to ESM2 (Lin et al., 2023), ProtoMech achieves state-of-the-art recovery of the original model’s performance, attaining 89% and 82% on protein family classification and function prediction tasks, respectively.
- We identify highly compact circuits that retain 79% (family) and 74% (function) of model performance while using only <1% of ProtoMech’s latent space. Steering along these circuits enables the generation of a diverse set of functional sequences, including the top-fitness variants in 71% of cases.
- By analyzing circuits associated with kinase activity, NADP+ binding, and GBI, we show that ProtoMech recovers computational pathways that align with known structural and functional motifs. To facilitate broader circuit discovery, we release ProtoMech as open-source software and provide accompanying visualization tools.¹

2. The ProtoMech Framework

In this section, we present the ProtoMech framework. ProtoMech is built around four interconnected components: (i) cross-layer transcoders (CLTs) for constructing a replacement model, (ii) a circuit discovery algorithm for identifying

computational pathways, (iii) a steering mechanism for manipulating these circuits, and (iv) visualization tools for interpreting the circuits.

2.1. Cross-layer Transcoders (CLTs)

CLTs extend standard transcoders by approximating the input–output mapping of each MLP layer as a function of the sparse latent variables derived from all preceding layers (Fig. 2a). This formulation replaces independent layerwise surrogates with a compositional model of inter-layer computation.

Let L be the number of transformer layers and d_{model} the hidden dimension of the pLM. For $\ell \in 1, \dots, L$, we denote by $\mathbf{x}^\ell \in \mathbb{R}^{d_{\text{model}}}$ the residual stream activation prior to the MLP block at layer ℓ . Sparsity in the latent space is enforced using a TopK activation function (Makhzani & Frey, 2013). Each residual stream activation is encoded into a latent vector $\mathbf{a}^\ell \in \mathbb{R}^{d_{\text{latent}}}$ via an encoder matrix:

$$\mathbf{a}^\ell = \text{TopK}(\mathbf{W}_{\text{enc}}^\ell (\mathbf{x}^\ell - \mathbf{b}_{\text{pre}}^\ell) + \mathbf{b}_{\text{enc}}^\ell), \quad (1)$$

where $\mathbf{W}_{\text{enc}}^\ell \in \mathbb{R}^{d_{\text{latent}} \times d_{\text{model}}}$ denotes the encoder matrix and $\mathbf{b}_{\text{pre}}^\ell \in \mathbb{R}^{d_{\text{model}}}$ is the corresponding bias term at layer ℓ . To regulate the number of active latent features, the TopK operator retains only the k largest-magnitude latent activations and sets all others to zero. To reconstruct the output of the MLP block at layer ℓ , denoted $\mathbf{y}^\ell \in \mathbb{R}^{d_{\text{model}}}$, CLTs employ decoder matrices that map latent representations from preceding layers to layer ℓ according to:

$$\hat{\mathbf{y}}^\ell = \sum_{\ell'=1}^{\ell} \mathbf{W}_{\text{dec}}^{\ell' \rightarrow \ell} \mathbf{a}^{\ell'} + \mathbf{b}_{\text{pre}}^\ell, \quad (2)$$

where $\hat{\mathbf{y}}^\ell \in \mathbb{R}^{d_{\text{model}}}$ denotes the reconstructed MLP output

¹Our code can be found at <https://github.com/amirgroup-codes/ProtoMech> and our visualizer is available at <https://protmech.github.io/>.

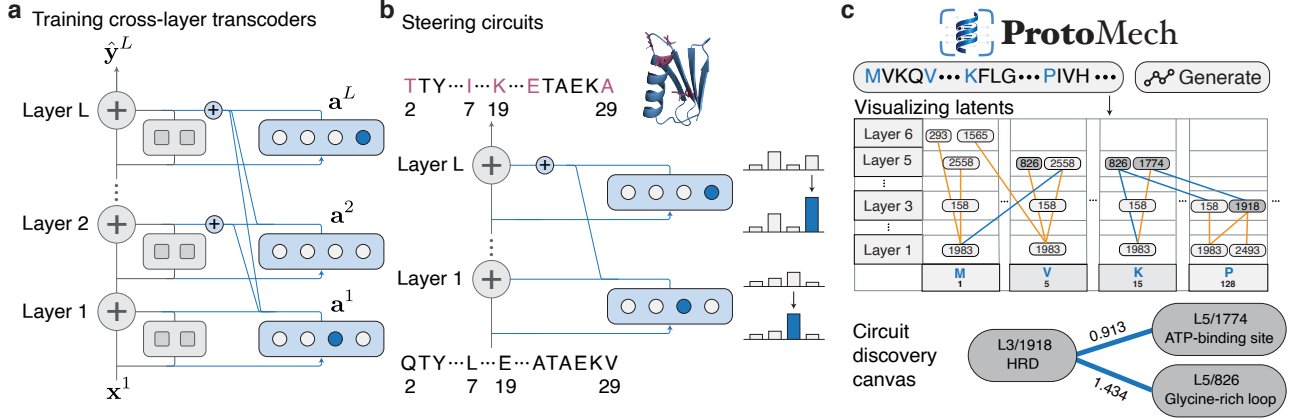


Figure 2. Overview of ProtoMech. **a**, Cross-layer transcoders (CLTs) form a replacement model that captures inter-layer computation by predicting each layer’s output from the sparse latent features of all preceding layers. **b**, Steering along identified circuits enables the design of protein variants with enhanced functional properties. **c**, Using the ProtoMech visualizer, we expose overlapping biological motifs previously hidden in ESM2.

at layer ℓ , and $\mathbf{W}_{\text{dec}}^{\ell' \rightarrow \ell}$ is a decoder matrix mapping latent features from layer ℓ' to layer ℓ . By requiring each reconstruction to depend on latent representations from all preceding layers, CLTs model the accumulation of computation across depth and thereby expose the inter-layer pathways that generate the model’s outputs.

We train CLTs by minimizing the mean-squared error between the original MLP output $\mathbf{y}^\ell$ and its reconstruction $\hat{\mathbf{y}}^\ell$ across all layers, $\mathcal{L}_{\text{MSE}} = \sum_{\ell=1}^L \|\mathbf{y}^\ell - \hat{\mathbf{y}}^\ell\|_2^2$. To mitigate the emergence of inactive (“dead”) latent units, we incorporate an auxiliary loss inspired by (Gao et al., 2025). Let $\mathbf{e}^\ell = \mathbf{y}^\ell - \hat{\mathbf{y}}^\ell$ denote the reconstruction residual at layer ℓ . The auxiliary loss is then defined as: $\mathcal{L}_{\text{aux}} = \sum_{\ell=1}^L \|\mathbf{e}^\ell - \hat{\mathbf{e}}^\ell\|_2^2$. Here, $\hat{\mathbf{e}}^\ell$ is obtained by decoding the top- k_{aux} latent activations in $\mathbf{a}^\ell$ using the decoder matrix $\mathbf{W}_{\text{dec}}^{\ell \rightarrow \ell}$, where k_{aux} is a hyperparameter. The full CLT training objective is then defined as

$$\mathcal{L}_{\text{CLT}} = \mathcal{L}_{\text{MSE}} + \alpha \mathcal{L}_{\text{aux}}, \quad (3)$$

where α controls the weight of the auxiliary loss. This joint objective enforces faithful reconstruction of MLP outputs while encouraging broad usage of the latent space, thereby increasing the number of interpretable latent features.

We train CLTs on the ESM2-8M model, for which $L = 6$ and $d_{\text{model}} = 320$. Following a procedure similar to (Adams et al., 2025), we train on 5 million protein sequences of length up to 1022 amino acids, randomly sampled from UniRef50 (Suzek et al., 2007). After training, the CLT functions as a replacement model for ESM2: at each layer, it substitutes the original MLP block with a sparse, interpretable latent representation that approximates the original computation (Fig. 1a). For architectural and optimization details, see Appendix A. To enable controlled comparison, we also train a PLT using the same hyperparameters and

sparsity constraint k as the CLT (see Appendix B).

2.2. Circuit Discovery

Leveraging the interpretable latent space induced by ProtoMech, we seek to recover the minimal subset of latent variables that governs task-specific computation in ESM2. While the CLT latent dimension is large, we posit that model behavior is mediated by a sparse collection of active latent features. Following (Dunefsky et al., 2024), we define a *circuit* as a set of latent variables whose joint activity is responsible for a specified model behavior.

Replacement Model. To identify such circuits, we first train a supervised probe on the final MLP output of ESM2, $\mathbf{y}^L$, to establish the performance of the original model on the target task. We then seek the minimal subset of ProtoMech latent variables capable of producing a reconstructed final MLP output, $\hat{\mathbf{y}}^L$, that recovers this performance (Fig. 1a).

To compute $\hat{\mathbf{y}}^L$, we adopt a replacement model strategy similar to that of (Ameisen et al., 2025). Specifically, during the forward pass, the original MLP blocks are replaced by the ProtoMech circuit, while the outputs of the attention heads are held fixed to those of the ground-truth ESM2 model. This hybrid formulation isolates the contribution of the MLP pathway while avoiding error accumulation from repeatedly reconstructing attention representations.

We observe that a fully recursive replacement, where both attention and MLP computations are derived from ProtoMech reconstructions at each layer, leads to substantial performance degradation due to compounding reconstruction errors (Ameisen et al., 2025). A comprehensive comparison of alternative replacement strategies, including prior approaches and new variants introduced here, is provided in Appendix C. We apply this replacement model framework